

Blending 2

Problem

- Find the optimal purchase, storage and production policy.

Sets

- Oils: $o = 1(VEG1), 2(VEG2), 3(OIL1), 4(OIL2), 5(OIL3)$.
- Months: $m = 1(Jan), \dots, 6(Jun)$

Parameters

- Sales price of final product: $SellPrice$.
- Purchase price of raw oil o in month m : $PurchaseCost_{m,o}$
- Hardness of raw oil o : $Hardness_o$
- Lower hardness: $LowerHardness$
- Upper hardness: $UpperHardness$
- Refinery capacity of V oil: $RefCapVeg$
- Refinery capacity of NV oil: $RefCapNonVeg$
- Initial storage of each oil: $InitialStorage$
- Final storage of each oil: $FinalStorage$
- Storage cost: $StorageCost$
- Storage capacity: $StorageCap$

Decision variables

- Amount of raw oil o bought in month m : $x_{m,o}$
- Amount of raw oil o used in month m : $y_{m,o}$
- Ultimo stock of raw oil o in month m : $s_{m,o}$ (IMPORTANT)

Model

Objective:

- Objective is profit to be maximized:

$$\begin{aligned} & SellPrice \sum_m \sum_o y_{m,o} \\ & - \sum_m \sum_o PurchaseCost_{m,o} \cdot x_{m,o} \\ & - \sum_m \sum_o StorageCost \cdot s_{m,o} \end{aligned}$$

Constraints:

- Lower hardness:

$$\sum_o Hardness_o \cdot y_{m,o} \geq LowerHardness \sum_o y_{m,o}, \forall m$$

- Upper hardness:

$$\sum_o Hardness_o \cdot y_{m,o} \leq UpperHardness \sum_o y_{m,o}, \forall m$$

- Refinery capacity for Vegetable oils:

$$\sum_{o=1}^2 y_{m,o} \leq RefCapVeg, \forall m$$

- Refinery capacity for Non-Vegetable oils:

$$\sum_{o=3}^5 y_{m,o} \leq RefCapNonVeg, \forall m$$

- Storage capacity:

$$s_{m,o} \leq StorageCap, \forall m, o$$

- Final storage:

$$s_{o,6} = FinalStorage, \forall o$$

- Material balance and initial storage using just one constraint, but with conditional parts:

$$s_{m,o} = s_{m-1,o} | (m > 1) + InitialStorage | (m = 1) + x_{m,o} - y_{m,o}, \forall m, o$$

- Positive buying, use and storage:

$$x_{m,o}, y_{m,o}, s_{m,o} \geq 0, \forall m, o$$

In the above model, the material balance constraint is modeled with a single constraint that has conditional parts. There exists other ways to model this as well such as:

Alternative 1: Splitting conditional into two constraints:

- Material balance for later months:

$$s_{m,o} = s_{m-1,o} + x_{m,o} - y_{m,o}, \forall m > 1, o$$

- Material balance first month:

$$s_{1,o} = InitialStorage + x_{1,o} - y_{1,o}, \forall o$$

Alternative 2: Modeling storage to be at the beginning of the month (PRIMO) instead of the end (ULTIMO):

- Initial condition ($m = 1$):

$$s_{1,o} = InitialStore, \forall o$$

- Balance going into following month:

$$s_{m+1,o} = s_{m,o} + x_{m,o} - y_{m,o}, m < 6, o$$

- End condition ($m = 6$):

$$FinalStore = s_{6,o} + x_{6,o} - y_{6,o}, \forall o$$

The full model in Julia/JuMP, available with the name

`Blending2.jl`

from the book web-site, is given below:

```

*****
# Blending Assignment, "Mathematical Programming Modelling" (42112)
*****

*****
# Intro definitions
using JuMP
using HiGHS
*****

```

```

*****

# INDEXES
Oils = ["VEG1" "VEG2" "OIL1" "OIL2" "OIL3"]
O = length(Oils)
Months = ["Jan" "Feb" "Mar" "Apr" "May" "Jun"]
M = length(Months)

# DATA
Hardness = [8.8 6.1 2.0 4.2 5.0] # hardness of the different oils
SellPrice = 150 # selling price for final product
RefCapVeg = 200
RefCapNonVeg = 250
LowerHardness = 3
UpperHardness = 6
PurchaseCost = [ # PurchaseCost[m,o]
                110 120 130 110 115;
                130 130 110 90 115;
                110 140 130 100 95;
                120 110 120 120 125;
                100 120 150 110 105;
                90 100 140 80 135
                ]
StorageCap = 1000
StorageCost = 5
InitialStore=500
FinalStore=500

*****

*****

# Model
blending2 = Model(HiGHS.Optimizer)

# how much buy of different oils
@variable(blending2, x[1:M,1:O] >= 0)

# how much to refine and sell
@variable(blending2, y[1:M,1:O] >= 0)

# how much is on storage of RAW-OIL in the end of the month
@variable(blending2, s[1:M,1:O] >= 0)

```

```

# Maximize profit
@objective(blending2, Max,
    sum(SellPrice*y[m,o] for m=1:M, o=1:O) -
    sum(PurchaseCost[m,o]*x[m,o] for m=1:M, o=1:O) -
    sum(StorageCost*s[m,o] for m=1:M, o=1:O))

# veg capacity
@constraint(blending2, [m=1:M],
    sum(y[m,o] for o=1:2) <= RefCapVeg)

# non-veg capacity
@constraint(blending2, [m=1:M],
    sum(y[m,o] for o=3:O) <= RefCapNonVeg)

# storage limit
@constraint(blending2, [m=1:M,o=1:O],
    s[m,o] <= StorageCap)

# end storage
@constraint(blending2, [o=1:O],
    s[M,o] == FinalStore)

# lower hardness
@constraint(blending2, [m=1:M],
    sum(Hardness[o]*y[m,o] for o=1:O) >= LowerHardness*sum(y[m,o] for o=1:O))

# upper hardness
@constraint(blending2, [m=1:M],
    sum(Hardness[o]*y[m,o] for o=1:O) <= UpperHardness*sum(y[m,o] for o=1:O))

# storage balance constraint
@constraint(blending2, [m=1:M,o=1:O],
    s[m,o] == x[m,o] - y[m,o] + (m>1 ? s[m-1,o] : InitialStore) )

print(blending2)

#####

#####

# solve
optimize!(blending2)

#####

#####

```

```

println("-----");
if termination_status(blending2) == MOI.OPTIMAL
    println("RESULTS:")
    println("Objective Value = $(objective_value(blending2))")

    for m = 1:M
        println("\n\nMonth: $(Months[m]) -----")
        println("Tons of final product sold:\t$(sum(value(y[m,o]) for o=1:O))")
        println("Hardnes of final product:\t$(sum(Hardness[o]*value(y[m,o]) for o=1:O)/sum(y[m,o]))")
        print("Raw oils refined and blended:\t")
        for o=1:O
            if value(y[m,o])>0
                print("$(round(value(y[m,o]),digits=1)) tons $(Oils[o]), ")
            end
        end
        print("\nRaw oils bought:\t\t")
        for o=1:O
            if value(x[m,o])>0
                print("$(round(value(x[m,o]),digits=1)) tons $(Oils[o]), ")
            end
        end
        print("\nRaw oils stored:\t\t")
        for o=1:O
            if value(s[m,o])>0
                print("$(round(value(s[m,o]),digits=1)) tons $(Oils[o]), ")
            end
        end
    end
else
    println(" No solution")
end
println("-----");
#####

#####
println("Successfull end of $(PROGRAM_FILE)")
#####

```

Comments to the Julia/JuMP program:

- Notice: In the Julia/JuMP code we used ULTIMO storage (storage for month m is what is in store at the end of the month, as opposed to the beginning).